



ASSIGNMENT 6.2

Evolucion robusta de la interfaz de S5

Estado explicito, gateway cacheado y UX operable

DURACION	MODALIDAD	PREREQUISITOS
2 horas	Parejas, refactor sobre snapshot	Assignment 6.1 y app de S5

1. Encargo

Refactorizar la app de S5 para soportar reruns de forma explicita, conservar el ultimo resultado, comunicar loading/success/error, reintentar fallos y registrar telemetria agregada sin payloads.

Regla de continuidad

El starter es un snapshot de S5: el formulario y el gateway ya estan hechos.

2. Punto de partida

- Los TODO nuevos estan en session.py, policies.py, presentation.py, controller.py y app.py.
- PredictionController no importa Streamlit; en S7 la misma frontera podra consumir HTTP.
- El resultado lento sigue siendo valido: above_target es una señal, no un error de contrato.

3. Material que recibes

- exercises/02-robust-streamlit/problem/starter/
- Tests de controller, presentation, gateway, session y telemetry
- Snapshot de S5 con sus once campos
- DemoGateway, UnavailableGateway y politicas proporcionadas

4. Tareas

01 Completa la sesion - Inicializa de forma idempotente last_state, telemetry y last_values; clear vuelve a idle sin borrar contadores.

02 Implementa politicas - Clasifica confianza low/medium/high y latencia within_target/above_target con limites explicitos.

03 Construye la vista - Traduce PredictionPayload a un view model con copy seguro, versiones y mensaje de latencia.

04 Implementa el controlador - Emite loading, mide latencia, valida la salida, traduce excepciones, crea request_id y registra agregados.

05 Cablea Streamlit - Usa cache_resource para el gateway, session_state para estado observable, placeholder/status para loading y botones de retry/clear.

06 Prueba cuatro recorridos - Exito, error de contrato, bundle ausente y respuesta lenta; verifica que no se muestran traceback ni features en telemetry.

5. Entrega

Evidencia	Que debe permitir comprobar
Diff de evolucion	Identifica que se conserva de S5 y que se añade en S6.
App avanzada	Resultado persistente, cache, loading, error, retry y clear funcionales.
Tests	Suite verde y casos de confianza/latencia comprobados.
Telemetria	Snapshot con contadores, errores, latencias y versiones, sin payload.
Evidencia visual	Capturas o registro de exito, fallo, retry y limpieza.

6. Criterios de aceptacion

- [] La app mantiene el formulario y el contrato de S5.
- [] El gateway se carga como recurso y no por cada rerun.
- [] Se observan idle, loading, success y error.
- [] Retry y clear funcionan; clear conserva telemetria.
- [] Confianza y latencia se comunican sin sobreafirmar.
- [] Los errores tienen recuperacion y request_id, sin traceback ni payload.

7. Rubrica orientativa

Criterio	Puntos
Sesion y cache	2
Políticas y view model	2
Controlador	3
App y UX	2
Evidencia	1

8. Comprobacion

```
cd modules/06-ux-model-consumption/exercises/02-robust-streamlit/problem/starter
```

```
uv sync
```

```
uv run pytest
```

```
uv run ruff check src tests
```

```
uv run ruff format --check src tests
```

```
uv run --with 'streamlit>=1.40,<2.0' streamlit run app.py
```

9. Notas y limites

- No reimplementes collect_values ni el contrato de S4.
- No uses cache_resource como cache global de respuestas por usuario.
- La solucion docente se consulta en el debrief, no para copiar TODOS.

Referencia en el repositorio

semana6/modules/06-ux-model-consumption/exercises/02-robust-streamlit/README.md
semana6/modules/06-ux-model-consumption/guides/class-2-workshop.md